# Changing the graphics engine on Windows

Airline Manager Pro · planning notes · 2 November 2026

## Short answer

Yes. The graphics engine can be changed without destroying the game. How big the job is depends on which engine it is changed to.

## How the game is built today

- The game is written in TypeScript. On the PC it runs in a Microsoft Edge app window; on Android in the system WebView.
- The 3D is drawn with **three.js** (MIT licence) through WebGL.
- About **three quarters of the code never touches the graphics engine**: the airline, money, flights, schedules, crews, the court, the simulator link (SimConnect), saves, menus and the map.
- The 3D drawing sits in about **19 files**: the 3D flight view, the cabin, the offices, the courtroom, the people (`src/graphics/human.ts`) and the airports. Only these would change.
- **Saves and player data are never at risk**: the engine only draws what the data says.
- All 3D is built in code (aircraft, people, offices, airports from published dimensions), not hand-made model files. This ports well: the same shapes are rewritten in the new engine's language rather than modelled again by hand.

## Options, from easiest to hardest

| Option | What changes | Rough time (human team) | Notes |
|---|---|---|---|
| three.js WebGPU renderer | Small changes to the renderer and some materials | Days to a couple of weeks | Better performance and effects; Android can keep WebGL; PC and Android keep sharing code. |
| Another web engine (Babylon.js, PlayCanvas) | The 3D files are rewritten | Weeks | Game, menus, data and the Android version stay as they are. |
| Full game engine for PC only (Unity, Unreal, Godot) | Every 3D scene rebuilt, plus a bridge so the engine and the game logic talk to each other | Months | PC and Android no longer share their 3D code. Check licences for commercial sale (see below). |

## How fast in our way of working

"Months" is what a small human team would need. With the code written in our working sessions, the code itself is more like weeks. What sets the pace is how often a version can be built, looked at and corrected:

- **The look needs rounds.** Every scene is built, run and checked from screenshots, then adjusted: lighting, materials, people and the cabin each take a few rounds to match today's quality.
- **Testing on your PC.** The simulator link and the gamepad only work on a real Windows PC with the simulator.

- **Where the engine runs.** Today everything is built and screenshotted in the working environment. Unity and Unreal are large, Windows-first editors, so builds would go through GitHub's Windows build machines: slower per round, but it works.

| Engine | Fit for this game | Rough time (working sessions) | Licence |
|---|---|---|---|
| **Godot** | Fastest: small, runs in the working environment, builds Windows versions from there | About 2–4 weeks | MIT: free, no royalties |
| **Unity** | Very doable: can be driven entirely from C# code; built on GitHub | About 3–6 weeks | Unity Personal free up to a revenue limit; paid plans above it. Check current terms. |
| **Unreal** | Hardest: very large engine, slow C++ builds, GitHub's standard machines struggle | Longest | Royalty on sales above a threshold. Check current terms. Its strength (photoreal hand-made assets) isn't how this game is made. |

These are rough estimates; the main uncertainty is how many rounds the visuals need to look right.

## Other engines worth knowing

Unity and Unreal are big commercial engines with a closed core (Unreal's source is readable but not open-licensed), large asset stores, and licence fees or royalties once a game earns enough. The engines below differ mainly in **licence** (most are fully open source with no fees), **size** (most are much smaller), **language** and **where they run**.

### Native engines (a separate Windows program)

| Engine | Language | Licence (commercial sale) | How it differs from Unity / Unreal |
|---|---|---|---|
| **Godot 4** | GDScript, C#, C++ | MIT: free, no royalties | Small (about 100 MB), open source, fast to start and build; builds Windows and Android from Linux. Graphics a step below Unreal's top end, but good PBR lighting, shadows and global illumination. |
| **Stride** (formerly Xenko) | C# | MIT: free | The closest open-source equivalent to Unity's way of working (C#, scenes, components), with strong PBR graphics. Windows-focused; smaller community; Android support is weaker. |
| **Flax Engine** | C#, C++ | Free; 4 % royalty only on earnings above $250,000 per quarter per game | Modern, high-quality renderer close to Unreal's look, with Unity-like C# scripting. Source available. Small community. |
| **O3DE** (Open 3D Engine, Linux Foundation; from Amazon Lumberyard) | C++, Lua, visual scripting | Apache 2.0 / MIT: free | AAA-style renderer, fully open source, very modular. Heavy (large download, long C++ builds, strong PC needed) and a small community: similar weight to Unreal without the ecosystem. |
| **CRYENGINE** | C++, C# | 5 % royalty on gross revenue above $5,000 a year per game | Strong outdoor and lighting visuals. The royalty starts almost immediately; small community; heavy. |
| **Bevy** | Rust | MIT / Apache 2.0: free | Code-only engine (no full editor yet), very fast, modern design. Still before version 1.0, so its API changes between releases. |
| **Wicked Engine** | C++, Lua | MIT: free | Small, very modern renderer (ray tracing, DX12/Vulkan), mostly one developer. Windows and Linux; little tooling. |

### Web engines (keep today's setup: Edge window on PC, WebView on Android)

| Engine | Language | Licence | How it differs |
|---|---|---|---|
| **three.js with the WebGPU renderer** (what we use now, newer renderer) | TypeScript / JavaScript | MIT: free | No rewrite: the same code and scenes with a faster, more modern renderer; WebGL kept as fallback on Android. |
| **Babylon.js** (Microsoft) | TypeScript | Apache 2.0: free | A full game engine on the web: physics, PBR, WebGPU, inspector and editor built in. More "engine" than three.js; same platforms as today. |
| **PlayCanvas** | JavaScript / TypeScript | Engine MIT: free (the online editor is an optional paid plan) | Very fast loading and small, strong on mobile; editor is online. |

Unity and Unreal stand apart by their size, tools and asset stores: Unreal for photoreal graphics with hand-made assets, Unity for the widest platform support and the largest store and community. Their costs: **Unity Personal** is free only while revenue and funding in the last 12 months are under $200,000; above that a paid seat (Unity Pro) is needed for each developer, at a yearly price per seat (check unity.com/pricing). The per-install "runtime fee" announced in 2023 was cancelled in 2024. **Unreal** takes 5 % of gross revenue above the first $1 million per product.

## Compared for Airline Manager Pro

What matters for this game: PC and Android sharing as much as possible; 3D built in code (not hand-made models); the menus and screens being web (HTML) today; the PC simulator link (SimConnect, Windows only, already handled by the PC bridge); building and checking in our working environment; licence fees on a paid game; and the visual quality ("ultra" cabin and offices).

Scores 1 (poor) to 5 (best) for this project:

| | three.js WebGPU | Babylon.js | Godot | Unity | Stride | Flax | Unreal | O3DE |
|---|---|---|---|---|---|---|---|---|
| Effort to switch (5 = least) | 5 | 4 | 3 | 2 | 2 | 2 | 1 | 1 |
| PC and Android keep shared code | 5 | 5 | 3 (one engine for both, but the menus must be rebuilt) | 3 (same) | 1 | 2 | 2 | 1 |
| Fits 3D built in code | 5 | 5 | 4 | 4 | 4 | 4 | 3 | 3 |
| Menus and screens stay as they are | 5 | 5 | 2 | 2 | 1 | 1 | 1 | 1 |
| Builds and screenshots in our working environment | 5 | 5 | 5 | 3 (via GitHub Windows builds) | 3 | 3 | 1 | 1 |
| Top visual quality possible | 3 | 4 | 4 | 4 | 4 | 5 | 5 | 5 |
| Performance on PC | 4 | 4 | 4 | 5 | 4 | 5 | 5 | 4 |

| | three.js WebGPU | Babylon.js | Godot | Unity | Stride | Flax | Unreal | O3DE |
|---|---|---|---|---|---|---|---|---|
| Licence cost for a paid game | 5 (free) | 5 (free) | 5 (free) | 3 (free under $200k a year, then per seat) | 5 (free) | 4 (4 % above $250k a quarter) | 3 (5 % above $1M) | 5 (free) |
| **Total (of 40)** | **37** | **37** | **30** | **26** | **24** | **26** | **21** | **21** |

The totals weigh every row equally. If top visual quality matters most, Flax, Unreal and O3DE climb; if keeping one game for PC and Android matters most, the web options stay far ahead.

### What the scores mean

- **Best value now: three.js WebGPU.** No rewrite, better speed and effects, both platforms keep the same code. The natural first step whatever happens later.
- **If a web engine with more built in is wanted: Babylon.js.** Same platforms, more engine features (physics, inspector), free.
- **If a native engine is wanted: Godot.** Free, small, builds both Windows and Android from our working environment, and suits 3D built in code. The menus would be rebuilt in Godot's own interface system, or the game keeps its web menus on PC with Godot drawing only the 3D.
- **If C# like Unity but free: Stride** (Windows-first) or **Flax** (better graphics, small royalty above $250,000 a quarter).
- **Unity:** a solid choice with the biggest community and store; costs start once the company earns over $200,000 a year.
- **Unreal and O3DE:** best for photoreal hand-made content, heavy to build and the most work to switch; not a good fit for how this game is made.

## Recommendation

1. **Next step: the three.js WebGPU renderer** (days to a couple of weeks, no risk to Android).
2. If a native engine is wanted later, prefer **Godot** (or **Unity** / **Flax**) over Unreal for this game.
3. Lower the risk: **port one scene first** (for example the cabin or the operations office), compare it side by side with today's version, then decide on the whole switch.
4. Until then, keep new 3D work behind the game's own small helpers (the people builder `Human.build`, the glow `AimGlow` and similar) rather than spreading three.js calls further, so a later switch has fewer, smaller pieces to rewrite.

## Sources


- Unity licensing in 2026 (Personal limit $200,000): https://enginesdatabase.com/blog/state-of-unity-licensing-in-2026/
- Unity runtime fee cancelled: https://unity.com/products/pricing-updates
- Unreal and Godot pricing in 2026: https://gamineai.com/blog/the-2026-state-of-game-engine-pricing-unity-unreal-godot-and-alternatives
- Flax Engine FAQ (4 % above $250,000 per quarter): https://flaxengine.com/faq/

- CRYENGINE licensing (5 % above $5,000 a year): https://www.cryengine.com/support/view/licensing

This is planning information. Engine licences and prices change; check each engine's current terms before deciding.